

tf.GradientTape Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/GradientTape

Record operations for automatic differentiation.

Function Signatures

```
tf.GradientTape( persistent=False,  
watch_accessed_variables=True )
```

Code Examples

```
tf.GradientTape(  
    persistent=False, watch_accessed_variables=True  
)
```

```
tf.GradientTape(  
    persistent=False, watch_accessed_variables=True  
)
```

```
x = tf.constant(3.0)  
with tf.GradientTape() as g:  
    g.watch(x)  
    y = x * x  
    dy_dx = g.gradient(y, x)  
    print(dy_dx)  
    tf.Tensor(6.0, shape=(), dtype=float32)
```

```
x = tf.constant(3.0)
```

```
with tf.GradientTape() as g:
```

```
dy_dx = g.gradient(y, x)
```

Documentation

Record operations for automatic differentiation.

View aliases

Main aliases

`tf.autodiff.GradientTape`

See

Migration guide for
more details.

`tf.compat.v1.GradientTape`

Used in the notebooks

- Introduction to gradients and automatic differentiation
 - Advanced automatic differentiation
 - Better performance with `tf.function`
 - TensorFlow basics
 - Effective Tensorflow 2
 - Deep Convolutional Generative Adversarial Network
 - DeepDream
 - pix2pix: Image-to-image translation with a conditional GAN
 - Neural style transfer
 - Custom training: walkthrough

Operations are recorded if they are executed within this context manager and at least one of their inputs is being "watched".

Trainable variables (created by `tf.Variable` or `tf.compat.v1.get_variable`, where `trainable=True` is default in both cases) are automatically watched. Tensors can be manually watched by invoking the `watch` method on this context manager.

For example, consider the function $y = x * x$. The gradient at $x = 3.0$ can be computed as:

GradientTapes can be nested to compute higher-order derivatives. For example,

By default, the resources held by a GradientTape are released as soon as GradientTape.gradient() method is called. To compute multiple gradients over the same computation, create a persistent gradient tape. This allows multiple calls to the gradient() method as resources are released when the tape object is garbage collected. For example:

By default GradientTape will automatically watch any trainable variables that are accessed inside the context. If you want fine grained control over which variables are watched you can disable automatic tracking by passing watch_accessed_variables=False to the tape constructor:

Note that when using models you should ensure that your variables exist when using watch_accessed_variables=False. Otherwise it's quite easy to make your first iteration not have any gradients:

Note that only tensors with real or complex dtypes are differentiable.

Methods

`## batch_jacobian`

[View source](#)

Computes and stacks per-example jacobians.

See wikipedia article

for the definition of a Jacobian. This function is essentially an efficient implementation of the following:

```
tf.stack([self.jacobian(y[i], x[i]) for i in range(x.shape[0])]).
```

Note that compared to `GradientTape.jacobian` which computes gradient of each output value w.r.t each input value, this function is useful when `target[i,...]` is independent of `source[j,...]` for $j \neq i$. This assumption allows more efficient computation as compared to `GradientTape.jacobian`. The output, as well as intermediate activations, are lower dimensional and avoid a bunch of redundant zeros which would result in the jacobian computation given the independence assumption.

Example usage:

```
## gradient
```

[View source](#)

Computes the gradient using operations recorded in context of this tape.

In addition to Tensors, gradient also supports RaggedTensors. For example,

jacobian

[View source](#)

Computes the jacobian using operations recorded in context of this tape.

See wikipedia
article

for the definition of a Jacobian.

Example usage:

[View source](#)

Clears all information stored in this tape.

Equivalent to exiting and reentering the tape context manager with a new tape. For example, the two following code blocks are equivalent:

This is useful if you don't want to exit the context manager for the tape, or can't because the desired reset point is inside a control flow construct:

stop_recording

[View source](#)

Temporarily stops recording operations on this tape.

Operations executed while this context manager is active will not be recorded on the tape. This is useful for reducing the memory used by tracing all computations.

For example:

[View source](#)

Ensures that tensor is being traced by this tape.

watched_variables

[View source](#)

Returns variables watched by this tape in order of construction.

`__enter__`

[View source](#)

Enters a context inside which operations are recorded on this tape.

`__exit__`

[View source](#)

Exits the recording context, no further operations are traced.